Deadlock Detection, Avoidance, and Recovery

Objectives: To understand the concept of deadlock in operating systems. To implement Banker's Algorithm for deadlock avoidance and a simple deadlock detection technique. To analyze resource allocation patterns and determine whether the system is in a safe or deadlock state.

Problem Statement: Implement Banker's Algorithm and deadlock detection techniques. Analyze resource allocation patterns and evaluate system performance under different deadlock scenarios.

Software and Hardware Requirements: Software: Ubuntu / WSL / Virtual Machine, VS Code, GCC Compiler, Linux commands Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory: Deadlock Deadlock is a situation in which two or more processes wait indefinitely for resources held by each other.

Example: P1 → Waiting for R2 R2 → Held by P2 P2 → Waiting for R1 R1 → Held by P1 Deadlock Avoidance Deadlock avoidance checks resource allocation before granting resources to ensure that the system remains in a safe state.

Banker's Algorithm is used for deadlock avoidance.

Banker's Algorithm

Banker's Algorithm checks whether the available resources are sufficient to satisfy the remaining requirements of processes. T

The main data used are: • Available: Resources currently available. • Maximum: Maximum resources required by each process. • Allocation: Resources currently allocated to each process. • Need: Remaining resources required. The Need matrix is calculated as: Need = Maximum − Allocation If all processes can complete in some order, the system is in a Safe State.

Deadlock Detection

Deadlock detection checks whether the system has already entered a deadlock state. If some processes cannot complete because the required resources are not available, those processes may be involved in a deadlock.

Deadlock Recovery After detecting a deadlock, the operating system can recover by terminating selected processes or releasing resources so that other processes can continue.

Conclusion: The concepts of deadlock detection and avoidance were studied and implemented. Banker's Algorithm was used to identify the safe state and safe sequence, while the deadlock detection algorithm was used to identify deadlocked processes. The experiment helped understand how proper resource allocation and management can prevent and detect deadlock conditions in an operating system. #include <stdio.h>

Here is a complete example of the Banker's Algorithm in action, assuming a system with 5 processes (P0 through P4) and 3 resource types (A, B, and C).

Process	Allocation (A,B,C)	Max (A,B,C)	Need (A,B,C)	Available (A,B,C)
P0	0, 1, 0	7, 5, 3	7, 4, 3	3, 3, 2 (Initial)
P1	2, 0, 0	3, 2, 2	1, 2, 2	
P2	3, 0, 2	9, 0, 2	6, 0, 0	
P3	2, 1, 1	2, 2, 2	0, 1, 1	
P4	0, 0, 2	4, 3, 3	4, 3, 1	

Understanding the Table Setup The **Need** column is calculated first using the formula Need = Max - Allocation. The **Available** column shows what the system currently has sitting idle (3 of A, 3 of B, and 2 of C). The algorithm now checks if it can satisfy the "Need" of any process using only what is "Available".

Step 1: Finding the first process The system scans the Need column from top to bottom.

- P0 needs (7,4,3), but we only have (3,3,2). P0 must wait.
- P1 needs (1,2,2). The system has (3,3,2). This is a match. P1 is granted the resources, finishes its task, and returns the resources it was already holding (Allocation) back to the system.*New Available = Current Available (3,3,2) + P1 Allocation (2,0,0) =* ***(5, 3, 2)***

Step 2: Finding the next process The system continues scanning.

- P2 needs (6,0,0). We have (5,3,2). P2 must wait.
- P3 needs (0,1,1). We have (5,3,2). This is a match. P3 runs and returns its allocated resources.*New Available = Current (5,3,2) + P3 Allocation (2,1,1) =* ***(7, 4, 3)***

Step 3: Continuing the sequence

- P4 needs (4,3,1). We have (7,4,3). This is a match. P4 runs and returns its allocated resources.*New Available = Current (7,4,3) + P4 Allocation (0,0,2) =* ***(7, 4, 5)***

Step 4: Checking waiting processes The system loops back to the processes that were forced to wait.

- P0 needs (7,4,3). We now have (7,4,5). This is a match. P0 runs and returns its allocated resources.*New Available = Current (7,4,5) + P0 Allocation (0,1,0) =* ***(7, 5, 5)***

Step 5: Finishing the final process

- P2 needs (6,0,0). We have (7,5,5). This is a match. P2 runs and returns its allocated resources.*New Available = Current (7,5,5) + P2 Allocation (3,0,2) =* ***(10, 5, 7)***

Because all processes were able to finish without getting permanently stuck, the system is in a **Safe State**. The safe sequence executed was **P1 -> P3 -> P4 -> P0 -> P2**.

```
int main() {
    int n, i, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Using normal 1D arrays (lists) instead of 2D matrices
    int alloc[n], max[n], need[n], f[n], ans[n];
    int avail;
    int ind = 0;

    for (i = 0; i < n; i++) {
        f[i] = 0; // Mark all as unfinished
    }

    printf("Enter the total Available resources in the system: ");
    scanf("%d", &avail);

    for (i = 0; i < n; i++) {
        printf("\nFor Process P%d:\n", i);
        printf("Currently allocated resources: ");
        scanf("%d", &alloc[i]);
        printf("Maximum resources required: ");
        scanf("%d", &max[i]);

        // Calculate Need (Need = Max - Allocation)
        need[i] = max[i] - alloc[i];
    }
```

```
  // Banker's Algorithm Logic (Single Resource)
  for (k = 0; k < n; k++) {
    for (i = 0; i < n; i++) {
      if (f[i] == 0) { // If process is not finished
        if (need[i] <= avail) { // If system has enough resources
          ans[ind++] = i;
          avail += alloc[i]; // Free up resources when done
          f[i] = 1; // Mark as finished
        }
      }
    }
  }
  // Check for Safe State
  int safe = 1;
  for (i = 0; i < n; i++) {
    if (f[i] == 0) {
      safe = 0;
      printf("\nSystem is NOT in a safe state. Deadlock detected!\n");
      break;
    }
  }
if (safe == 1) {
    printf("\nSystem is in a SAFE state.\nSafe Sequence is: ");
    for (i = 0; i < n - 1; i++) {
      printf(" P%d ->", ans[i]);
    }
    printf(" P%d\n", ans[n - 1]);
  }

  return 0;
}
```

Input for safe

Enter number of processes: **3** Enter the total Available resources in the system: **2**

For Process P0: Currently allocated resources: **2** Maximum resources required: **5**

For Process P1: Currently allocated resources: **3** Maximum resources required: **7**

For Process P2: Currently allocated resources: **2** Maximum resources required: **4**

 Process P2 needs 2 more resources. Since the system has 2 available, P2 runs first. When it finishes, it returns its resources, making the total available 4.

 Next, Process P0 needs 3 resources. The system now has 4, so P0 runs and returns its resources. The total available becomes 6.

 Finally, Process P1 needs 4 resources. The system has 6, so P1 runs.

 The output will show a SAFE state with the sequence: **P2 -> P0 -> P1**

Input for not safe

Enter number of processes: **3** Enter the total Available resources in the system: **2**

For Process P0: Currently allocated resources: **1** Maximum resources required: **3**

For Process P1: Currently allocated resources: **2** Maximum resources required: **6**

For Process P2: Currently allocated resources: **2** Maximum resources required: **7**

 Process P0: Needs 2 more resources (Max 3 - Alloc 1). The system has 2 available, so P0 runs successfully. When it finishes, it returns its 1 allocated resource to the system. The total available is now **3**.

 Process P1: Needs 4 more resources (Max 6 - Alloc 2). The system only has 3 available, so P1 cannot run. It must wait.

 Process P2: Needs 5 more resources (Max 7 - Alloc 2). The system only has 3 available, so P2 cannot run. It must wait.

 The system is now stuck. P1 and P2 are waiting indefinitely for resources that will never be released.

 The output will show: **System is NOT in a safe state. Deadlock detected!**

Steps to run :

Ctrl + Alt + T

bankers.c:nano bankers.c

Ctrl + Shift + V

Ctrl + O

Enter

Ctrl + X

bankers:gcc bankers.c -o bankers

./bankers